

Callbacks, promesas y async/await: análisis comparativo del modelo de ejecución de JavaScript sobre un simulador de eventos temporizados

Juan Sebastián García Valderrama
Facultad de Ingeniería
Universidad de San Buenaventura
Bogotá, Colombia

John Alejandro Martín Galvis
Facultad de Ingeniería
Universidad de San Buenaventura
Bogotá, Colombia

Gabriela Rodríguez Lara
Facultad de Ingeniería
Universidad de San Buenaventura
Bogotá, Colombia

Resumen—Este informe documenta el desarrollo del laboratorio “JavaScript Avanzado”, cuyo objetivo es evaluar la capacidad de razonar sobre el modelo de ejecución de JavaScript (pila de llamadas, cola de macrotarefas y cola de microtarefas) a través de un simulador de ocho avisos temporizados, implementado de tres formas equivalentes: callbacks anidados, promesas encadenadas y `async/await`. Se comparan técnicamente los tres estilos, se predice y reconcilia el orden de ejecución observado, se procesa el arreglo de resultados con métodos funcionales (`reduce`, `filter`, `map` y `find`), y se diseña un caso límite propio -una lectura corrupta de un sensor que entrega un objeto `Error` sin que nada en la cadena lo valide- para poner a prueba el modelo de ejecución y el procesamiento funcional bajo datos inválidos. Los resultados muestran que el encadenamiento secuencial fuerza un orden de llegada idéntico e independiente de los retrasos individuales, a costa de una latencia acumulada creciente, y que ninguno de los tres estilos asíncronos protege por sí solo contra una promesa que se resuelve “exitosamente” con un valor inválido.

Index Terms—JavaScript, bucle de eventos, promesas, `async/await`, programación asíncrona, pila de llamadas, cola de macrotarefas, cola de microtarefas, métodos funcionales de arreglos.

I. INTRODUCCIÓN

El presente laboratorio, propuesto por el docente Ing. Sergio Alberto Reyes Gómez en la asignatura de JavaScript Avanzado de la Facultad de Ingeniería de la Universidad de San Buenaventura, tiene como objetivo evaluar la capacidad del estudiante para razonar sobre el modelo de ejecución de JavaScript, estructurar información mediante objetos y procesar colecciones de datos con métodos funcionales de arreglos, más allá de la simple escritura de código funcional [1].

Para ello se construyó un simulador que “dispara” ocho avisos, cada uno con un tiempo de espera distinto, y que se implementó de tres formas sintácticamente distintas pero semánticamente equivalentes: callbacks anidados, promesas encadenadas y `async/await`. Sobre el arreglo de resultados de cada implementación se aplicaron cuatro operaciones funcionales, y finalmente se diseñó un caso límite propio para poner a prueba tanto el modelo de ejecución como el procesamiento funcional bajo una condición de datos inválidos no contemplada en el enunciado original.

Este documento sigue el hilo de los seis pasos del laboratorio: diseño del simulador (Sección II), comparación técnica de los tres estilos (Sección III), predicción y reconciliación del orden de ejecución (Sección IV), procesamiento funcional del arreglo de resultados (Sección V), diseño y explicación del caso límite (Sección VI), y el diagrama del modelo de ejecución correspondiente a dicho caso límite (Sección VII).

II. DISEÑO DEL SIMULADOR

Cada uno de los ocho avisos se representa como un objeto con cuatro atributos mínimos: `eventName` (identificador), `eventType` (categoría), `scheduledTime` (instante en que se programó que apareciera, calculado como una marca de referencia `refTime` capturada una única vez al inicio del programa, más un retraso propio de cada evento) y `realTime` (instante en que efectivamente se resolvió, obtenido con `Date.now()`). El Listado 1 muestra la definición de `eventOne` en su forma basada en promesas; las siete funciones restantes siguen exactamente el mismo patrón, cambiando únicamente el nombre, la categoría y el retraso.

```
1 const EVENT_DELAYS_MS = {
2   EVENT_ONE: 480,
3   // ... EVENT_TWO .. EVENT_EIGHT
4 };
5
6 function eventOne() {
7   return new Promise((resolve) => {
8     setTimeout(() => {
9       resolve({
10         eventName: 'eventOne',
11         eventType: 'aviso_largo',
12         scheduledTime: refTime + EVENT_DELAYS_MS.EVENT_ONE,
13         realTime: Date.now(),
14       });
15     }, EVENT_DELAYS_MS.EVENT_ONE);
16   });
17 }
```

Listing 1. Patrón de definición de un evento (AsyncImpl.js).

Los ocho retrasos elegidos (480, 650, 260, 220, 720, 400, 180 y 560 ms para `eventOne` a `eventEight`, respectivamente) se escogieron deliberadamente sin un orden ascendente: en particular, el retraso de `eventThree` (260 ms) es menor que el de `eventTwo` (650 ms), lo que introduce a propósito una inversión que la Sección V explota para

ejercitar el método `find`. Tras la retroalimentación entre los tres integrantes del grupo, estos ocho valores se extrajeron a un objeto de constantes con nombre, `EVENT_DELAYS_MS`, en cada uno de los seis archivos que los usa: los valores numéricos en sí no cambiaron, pero dejaron de aparecer como literales sueltos (“números mágicos”) dentro de cada `setTimeout`, lo que mejora la trazabilidad de dónde se define cada retraso y evita tener que modificarlo en dos lugares distintos (el `scheduledTime` y el propio `setTimeout`) si se decidiera cambiarlo.

Con el fin de no duplicar innecesariamente la lógica de análisis entre las tres implementaciones -algo que la lógica de negocio no diferencia entre estilos, a diferencia de la orquestación de los propios eventos, que sí es el objeto de comparación-, el post-procesamiento del Paso 4 (`reduce`, `filter+map` y `find`) se extrajo a un único módulo, `ProcessResults.js`, que las tres implementaciones importan y ejecutan sobre su propia bitácora al finalizar. Las ocho funciones de evento, en cambio, sí se mantienen duplicadas en cada archivo: ahí reside precisamente la diferencia sintáctica que el laboratorio pide comparar.

III. COMPARACIÓN TÉCNICA DE LOS TRES ESTILOS

III-A. Manejo de errores

En **callbacks anidados**, no existe un mecanismo del lenguaje que propague automáticamente un error entre niveles: cada función recibe lo que sea que el nivel anterior le haya pasado a su callback, sin distinguir sintácticamente entre un dato válido y uno inválido, y `CallbackImpl.js` no incorpora ningún mecanismo adicional para ello, precisamente porque el estilo no ofrece un punto único donde centralizarlo. En **promesas encadenadas**, sí existe ese punto único: tras la revisión del grupo, `PromiseImpl.js` incorporó un `.catch()` al final de toda la cadena, que efectivamente intercepta cualquier `reject(...)` o excepción síncrona lanzada dentro de algún *executor*. En **async/await**, `AsyncImpl.js` incorporó de forma análoga un `try/catch` que envuelve las ocho llamadas a `await`. Ambas adiciones son mejoras genuinas -antes no existía ningún manejo explícito de errores en ninguna de las tres implementaciones- pero comparten la misma limitación estructural: solo se activan si algún eslabón efectivamente *rechaza* su promesa. El caso límite de la Sección VI pone precisamente esa limitación a prueba sobre el código ya corregido: al resolver una promesa con un objeto `Error` en lugar de rechazarla, ni el nuevo `.catch()` de `PromiseImpl.limitCase.js` ni el nuevo `try/catch` de `AsyncImpl.limitCase.js` llegan a activarse, porque desde la perspectiva del motor de promesas la operación terminó exitosamente. Es decir, agregar manejo de errores convencional fue una mejora necesaria para los errores que sí se propagan como rechazo, pero no basta, por sí sola, contra los que no.

III-B. Legibilidad del orden de ejecución

En **callbacks anidados**, el orden de ejecución coincide con el orden de anidación visual del código: nuestra implemen-

tación en `CallbackImpl.js` llega a nueve niveles de indentación para los ocho eventos, un síntoma reconocible de lo que la comunidad llama *pyramid of doom*, sin importar qué tan descriptivos sean los nombres de las variables. En **promesas encadenadas**, el orden se aplanó en una lista de llamadas a `.then()`, más fácil de seguir que el anidamiento; en la versión revisada, además, cada parámetro de callback pasó de un nombre corto (`e1`, `e2`, ...) a uno descriptivo (`eventOneResult`, `eventTwoResult`, ...), lo que elimina la necesidad de rastrear qué representa cada uno consultando el eslabón anterior. En **async/await**, la secuencia se lee de arriba hacia abajo como código síncrono ordinario -ocho líneas casi idénticas de `register.push(await eventN())`-, siendo la forma más legible de las tres para expresar una secuencia estrictamente ordenada como la de este simulador.

III-C. Recomendación técnica

Para el desarrollo de backend con enfoque asíncrono recomendamos **async/await** como estilo por defecto: combina la legibilidad de código síncrono con `try/catch` nativo para el manejo de errores, sin sacrificar nada del modelo de promesas subyacente (de hecho, `await` es azúcar sintáctico sobre `.then()` [2]). Las promesas encadenadas siguen siendo útiles cuando la secuencia de pasos se construye dinámicamente en tiempo de ejecución (por ejemplo, a partir de un arreglo de tareas cuyo tamaño no se conoce de antemano), un escenario donde la sintaxis lineal de `await` no siempre es la más natural. Los **callbacks anidados**, por último, consideramos que deben reservarse a un rol pedagógico -entender el mecanismo subyacente- más que a código de producción.

IV. PREDICCIÓN Y RECONCILIACIÓN

Predicción (antes de ejecutar). A pesar de que los ocho eventos tienen retrasos distintos y no ordenados (Sección II), predijimos que la bitácora final de las tres implementaciones mostraría siempre los eventos en el orden de declaración (`eventOne` a `eventEight`), nunca ordenados por magnitud de retraso. La razón es que en las tres implementaciones cada evento sólo se invoca -y por lo tanto su `setTimeout` sólo se registra en la cola de macrotareas- una vez que el evento anterior ya se resolvió: nunca hay dos temporizadores compitiendo simultáneamente en la cola de macrotareas, así que no puede haber una carrera que reordene la llegada. También predijimos que `eventThree` sería detectado como el primer evento “fuera de orden” por el algoritmo de la Sección V, dado que su `scheduledTime` (260 ms de retraso) es menor que el de `eventTwo` (650 ms), que lo precede en la bitácora. Finalmente, predijimos que la latencia observada (`realTime - scheduledTime`) crecería monótonamente evento a evento, porque el instante real de cada evento acumula la *suma* de todos los retrasos anteriores (al ser estrictamente secuenciales), mientras que su `scheduledTime` sólo refleja su propio retraso individual sobre `refTime`.

Reconciliación (después de ejecutar). La Tabla I resume una ejecución real de `AsyncImpl.js`, ya con las

Cuadro I
RETRASO PROGRAMADO Y LATENCIA OBSERVADA POR EVENTO
(EJECUCIÓN REAL DE ASYNCIMPL.JS).

Evento	Retraso (ms)	scheduledTime (offset)	Latencia (ms) (realTime-scheduledTime)
eventOne	480	480	1
eventTwo	650	650	482
eventThree	260	260	1133
eventFour	220	220	1394
eventFive	720	720	1615
eventSix	400	400	2336
eventSeven	180	180	2737
eventEight	560	560	2918

constantes `EVENT_DELAYS_MS` y el `try/catch` incorporados por el grupo (los resultados de `CallbackImpl.js` y `PromiseImpl.js` son cualitativamente idénticos, y los ocho retrasos en sí no cambiaron respecto a la versión anterior). Los tres archivos confirmaron el orden de declaración sin excepción, confirmaron a `eventThree` como el primer evento fuera de orden, y confirmaron el crecimiento monótono de la latencia: de 1 ms en `eventOne` a 2918 ms en `eventEight`. El resultado coincidió exactamente con lo predicho; no hubo desajuste que explicar.

V. PROCESAMIENTO FUNCIONAL DEL ARREGLO DE RESULTADOS

El Listado 2 muestra las tres operaciones sobre la bitácora (`register`), tal como quedaron implementadas en `ProcessResults.js` (renombrado de `procesarResultados.js` durante la revisión del grupo, junto con sus variables internas, para mantener el código consistentemente en inglés).

```

1  const totalLatency = register.reduce(
2    (accum, current) =>
3      accum + (current.realTime - current.scheduledTime),
4      0
5  );
6  const averageLatency = totalLatency / register.length;
7
8  const deviationEventIds = register
9    .filter(e => (e.realTime - e.scheduledTime) > 100)
10   .map(e => e.eventName);
11
12  let maxScheduledSoFar = -Infinity;
13  const firstOutOfOrderEvent = register.find((e) => {
14    if (e.scheduledTime < maxScheduledSoFar) return true;
15    maxScheduledSoFar = Math.max(maxScheduledSoFar, e.
16      scheduledTime);
17    return false;
18  });

```

Listing 2. Post-procesamiento funcional compartido (`ProcessResults.js`, resumido).

Latencia promedio (`reduce`). La ejecución reportó una latencia promedio de 1577 ms. Este número no es un promedio de los retrasos que elegimos (que promedian 434 ms): es el promedio de *cuánto tardó de más* cada evento en llegar respecto a su propia marca individual, y como se explicó en la Sección IV, ese exceso crece con cada evento porque el encadenamiento secuencial obliga a que cada uno espere a que termine el anterior. En otras palabras, la latencia promedio mide el costo de haber encadenado ocho esperas una detrás

de otra en lugar de dispararlas en paralelo, no la duración de ninguna espera individual.

Identificadores con desviación (`filter+map`). Con un umbral de 100 ms, definido por nosotros por ser lo suficientemente pequeño para capturar cualquier evento afectado por la acumulación secuencial, el resultado fue `[eventTwo, eventThree, eventFour, eventFive, eventSix, eventSeven, eventEight]`: los siete eventos posteriores al primero. Sólo `eventOne` queda fuera, porque al ser el primero en dispararse, su `realTime` coincide casi exactamente con su `scheduledTime` (diferencia de 1 ms en la Tabla I).

Primer evento fuera de orden (`find`). El resultado fue `eventThree`, tal como se predijo en la Sección IV.

Justificación de los métodos elegidos. `forEach` se descartó para las tres operaciones porque siempre retorna `undefined`: usarlo habría obligado a mutar una variable externa al callback para acumular cualquier resultado, un efecto colateral innecesario cuando lo que se necesita es un valor derivado del arreglo. `reduce` es la elección correcta cuando el objetivo es condensar todo el arreglo en un único valor acumulado (la suma de latencias) sin mutar nada fuera de su propio callback. `filter` es la elección correcta para *seleccionar* un subconjunto sin transformar cada elemento, mientras que `map` es la elección correcta para *transformar* cada elemento del subconjunto ya filtrado en el dato que realmente interesa reportar (el identificador, no el objeto completo). Ninguno de los dos muta el arreglo original: ambos retornan arreglos nuevos, lo cual es deseable cuando `register` debe seguir siendo la única fuente de verdad durante todo el análisis. Por último, `find` y no `filter` es la elección correcta para el tercer requerimiento porque el enunciado pide explícitamente *el primer* evento fuera de orden, no todos los que califiquen; `find` se detiene apenas encuentra ese elemento, mientras que `filter` habría recorrido el arreglo completo para devolver una lista que no fue lo que se pidió.

VI. DISEÑO Y EXPLICACIÓN DEL CASO LÍMITE

VI-A. Diseño y justificación

El caso límite propio, al que llamamos *lectura corrupta del sensor*, agrega un noveno aviso (`eventNine`, renombrado de `eventoNueve` durante la revisión del grupo) justo después de `eventThree`. A diferencia de los ejemplos sugeridos en el enunciado (timestamps idénticos, una excepción no capturada dentro de un `.then`, o un `await` sobre una promesa que nunca resuelve), nuestro caso combina dos observaciones propias: primero, que envolver una operación en `new Promise(...)` no garantiza que el valor con el que se resuelve tenga la forma esperada; y segundo, que `resolve(...)` no valida en absoluto el tipo del valor que recibe. El Listado 3 muestra su definición.

```

function eventNine() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(new Error(
        'eventoNueve: ¿no se pudo confirmar' +
        ' la lectura del sensor'
      ));
    }, 100);
  });
}

```

```

7     });
8     }, EVENT_DELAYS_MS.EVENT_NINE);
9     });
10  }

```

Listing 3. Evento límite: resuelve con un `Error` en lugar de rechazar (`PromiseImpl.limitCase.js`).

En la cadena de consumo (Listado 4), a propósito *no* se valida si el valor recibido es una instancia de `Error` antes de insertarlo en la bitácora, para observar qué hace el modelo de ejecución -y, en cascada, el procesamiento funcional de la Sección V- cuando nadie se detiene a comprobar la forma de un dato asíncrono. Esto se mantuvo así deliberadamente incluso después de que `AsyncImpl.js` incorporara el `try/catch` descrito en la Sección III: como se verá en la Sección VI-C, ese `try/catch` también envuelve a `eventNine()` en `AsyncImpl.limitCase.js`, y aun así no intercepta nada.

```

1  try {
2    // ...
3    register.push(await eventNine()); // sin validar
4    register.push(await eventFour());
5    // ...
6    processResults(register);
7  } catch (error) {
8    console.error('Error_durante_la_ejecucion_de_eventos:',
9      error);

```

Listing 4. Consumo sin validar, ya dentro del bloque `try` incorporado por el grupo (`AsyncImpl.limitCase.js`).

Consideramos que este caso no es trivial porque, a diferencia de un `throw` sin capturar -que detiene la ejecución de forma ruidosa y evidente-, este produce una corrupción *silenciosa*: el programa termina normalmente, sin ninguna excepción visible y sin entrar jamás al bloque `catch`, pero con un dato inválido incrustado en el resultado final.

VI-B. Predicción (antes de ejecutar)

Predijimos que la bitácora final tendría longitud 9, con el objeto `Error` en la posición 3 (índice base 0). Sobre el procesamiento del Paso 4, predijimos que `reduce` devolvería `NaN`, porque `current.realTime` y `current.scheduledTime` son `undefined` en un objeto `Error`, y `undefined - undefined` evalúa a `NaN`; una vez que el acumulador de `reduce` se contamina con `NaN`, permanece así para siempre porque `NaN` sumado a cualquier número sigue siendo `NaN`. Predijimos que `filter+map` excluiría el `Error` sin lanzar ninguna excepción, porque la comparación `NaN > 100` evalúa siempre a `false`. Y predijimos que `find` no se vería afectado, porque ya encuentra a `eventThree` -en la posición 2- antes de llegar al `Error` en la posición 3, y `find` se detiene en el primer acierto.

VI-C. Ejecución y reconciliación

Los tres estilos (callbacks, promesas y `async/await`) reprodujeron exactamente el comportamiento predicho. La bitácora final tuvo longitud 9 en los tres casos, con el objeto `Error` en la posición 3 -al imprimirlo, Node.js mostró incluso su traza de pila completa, confirmando que se trata de una instancia real de `Error` y no de un objeto con esa apariencia-. La latencia promedio reportó `NaN` en los tres archivos. La lista de identificadores con desviación excluyó al `Error` en silencio,

devolviendo exactamente los mismos siete eventos que en la Tabla I. Y `find` siguió devolviendo a `eventThree`, sin ningún cambio respecto al Paso 4 sin el caso límite.

Adicionalmente, y esto es lo que este caso límite ponía a prueba de forma más específica sobre la versión revisada del código: ni el `.catch()` de `PromiseImpl.limitCase.js` ni el `try/catch` de `AsyncImpl.limitCase.js` se activaron en ninguna ejecución. La consola nunca mostró el mensaje '`Error durante la ejecución...`' que ambos manejadores imprimen; en su lugar, `processResults(register)` corrió con total normalidad dentro del mismo bloque `try`, sobre un arreglo que ya contenía el `Error` sin detectar. Esto confirma, con código defensivo real y no solo en teoría, que agregar manejo de errores convencional no es suficiente contra este tipo de falla.

VI-D. Explicación causal

El resultado se explica enteramente por cómo JavaScript trata el tipo de un valor de cumplimiento de una promesa: `resolve(valor)` acepta cualquier valor, sin restricción de forma ni de tipo [4]. Ni la cadena de promesas, ni `async/await`, ni los callbacks anidados verifican en ningún punto que ese valor tenga las propiedades `eventName`, `eventType`, `scheduledTime` y `realTime` que el resto del programa asume: simplemente lo entregan, tal cual, a quien haya escrito la continuación. La corrupción se refleja en el arreglo de objetos-evento como un elemento de un tipo distinto al resto -un `Error` en lugar de un objeto plano- insertado exactamente en la posición donde `eventNine` fue invocado, sin desplazar ni omitir a ningún otro evento.

Su reflejo en el procesamiento del Paso 4 varía por operación, y esa variación es en sí misma la lección más importante del caso: `reduce` propaga el daño globalmente porque acumula un único valor numérico y `NaN` es contagioso en aritmética; `filter` lo excluye de forma benigna porque toda comparación relacional contra `NaN` (o contra `undefined`, que se convierte a `NaN` al operar numéricamente) evalúa a `false`; y `find` resultó inmune únicamente por la posición relativa de los elementos -encontró su respuesta antes de llegar al dato corrupto-, no porque el algoritmo esté protegido: si `eventNine` se hubiera colocado antes de `eventThree` en la secuencia, `maxScheduledSoFar` se habría contaminado con `NaN` en el mismo paso (`Math.max` con un `undefined` entre sus argumentos devuelve `NaN`), y ninguna comparación posterior habría podido volver a marcar un evento como fuera de orden. En ambos escenarios, el programa nunca lanza una excepción ni se detiene: produce silenciosamente una respuesta incorrecta, que es precisamente el tipo de falla más difícil de detectar en un sistema en producción.

VII. DIAGRAMA DEL MODELO DE EJECUCIÓN

La Figura 1 traza, paso a paso, la pila de ejecución, la cola de microtarefas y la cola de macrotarefas de `AsyncImpl.limitCase.js` durante el segmento crítico del caso límite: desde que se dispara el temporiza-

dor de `eventThree` hasta que `processResults()` corra sobre el arreglo ya corrompido. Se eligió la versión `async/await` porque `await` es semánticamente equivalente a encadenar `.then()` [5], por lo que el mismo diagrama describe igualmente el comportamiento de `PromiseImpl.limitCase.js`. El diagrama también deja ver por qué el `try/catch` agregado en la revisión del grupo (Sección III) no cambia en nada esta traza: como se ve en T2, T4 y T6, todo el segmento ocurre *dentro* del bloque `try` (líneas 124-143), pero ninguno de sus pasos lanza una excepción que ese bloque pueda interceptar.

Cada columna T_n representa un turno completo del bucle de eventos: o bien la ejecución de una macro-tarea (un callback de `setTimeout` ya vencido), o bien el drenado de una micro-tarea (la continuación de un `await`). La cola de micro-tareas aparece vacía en las seis columnas porque, en este simulador, cada micro-tarea generada se ejecuta de inmediato -antes de que se tome la siguiente instantánea-, nunca se acumula más de una en espera.

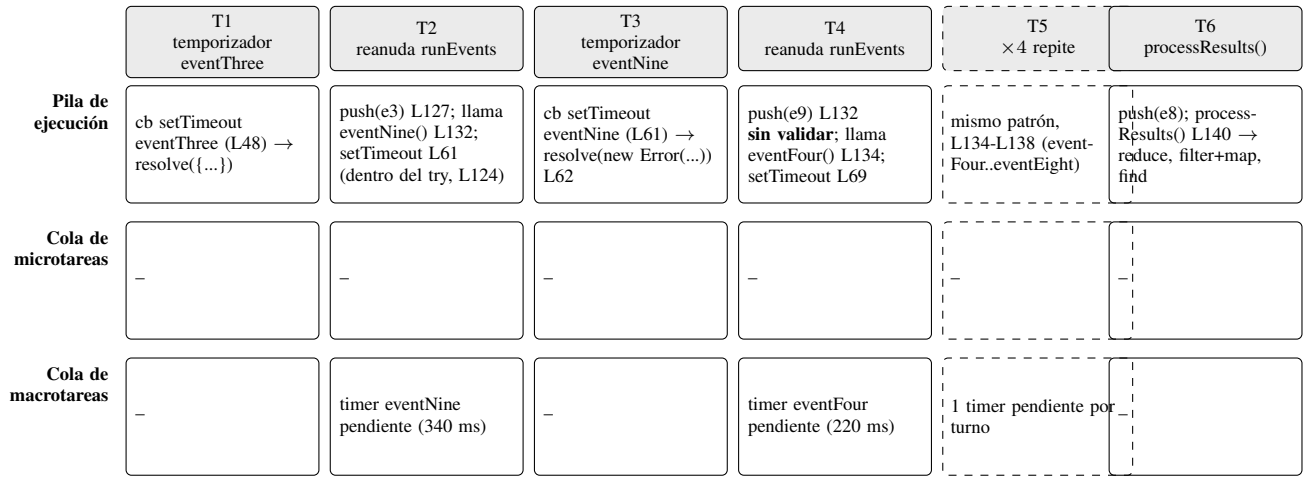


Figura 1. Pila de ejecución, cola de microtarefas y cola de macrotareas durante el caso límite “lectura corrupta del sensor” (AsyncImpl.limitCase.js). Las referencias L_n indican el número de línea del archivo fuente; todo el segmento T1-T6 transcurre dentro del bloque `try` agregado por el grupo (L124-143).

El punto crítico del diagrama es T3: el mecanismo que ejecuta el callback del temporizador de `eventNine` es idéntico, línea por línea del motor de JavaScript, al que ejecutó el de `eventThree` en T1. Ningún componente del modelo de ejecución -ni la pila, ni ninguna de las dos colas- distingue entre resolver con un objeto-evento válido y resolver con un `Error`: esa distinción sólo existe, o debería existir, en el código de la aplicación, no en el bucle de eventos. Tampoco el bloque `try` que envuelve todo el segmento distingue nada aquí: un `try/catch` solo reacciona ante una excepción lanzada o una promesa rechazada, y en T1-T6 no ocurre ninguna de las dos cosas. La corrupción permanece invisible para el modelo de ejecución hasta T6, donde `processResults()` corre de forma completamente síncrona -dentro de esa misma microtarea final, y todavía dentro del `try`- sobre el arreglo completo, y es recién ahí, al ejecutar `reduce`, donde el dato inválido empieza a propagar `NaN`.

VIII. CONCLUSIONES

El encadenamiento secuencial -presente por igual en callbacks anidados, promesas encadenadas y `async/await`- garantiza que el orden de llegada de los ocho eventos coincida siempre con el orden de declaración, sin importar la magnitud de sus retrasos individuales, porque nunca compete más de un temporizador a la vez en la cola de macrotareas. Ese orden garantizado tiene un costo: la latencia observada crece de forma acumulada evento a evento, y la latencia promedio reportada por `reduce` termina midiendo ese costo de secuenciación, no la duración de ninguna espera individual.

De los tres estilos comparados, `async/await` ofrece la mejor combinación de legibilidad y manejo de errores para código de backend con enfoque asíncrono, aunque ninguno de los tres protege, por sí solo, contra una promesa que se resuelve “exitosamente” con un valor de forma incorrecta. Esto no quedó solo en el terreno de la hipótesis: tras incorporar deliberadamente `.catch()` a `PromiseImpl.js` y `try/catch` a `AsyncImpl.js` durante la revisión del grupo

-mecanismos de manejo de errores genuínos, que sí habrían interceptado un `reject(...)` o un `throw`-, el caso límite diseñado demostró que ninguno de los dos se activa cuando el problema no es que la promesa falle, sino que tenga éxito con el dato equivocado: un `Error` entregado sin validar a través de callbacks, promesas y `await` -con o sin manejo de errores explícito- sin disparar ninguno de sus respectivos mecanismos. Los métodos funcionales de arreglo (`reduce`, `filter`, `map`, `find`) reaccionaron de forma previsible ante ese dato corrupto gracias a la semántica de `NaN` y `undefined` en JavaScript, pero esa misma previsibilidad es un arma de doble filo: el programa nunca lanzó una excepción, sólo produjo silenciosamente una respuesta numérica incorrecta. La validación explícita de la forma de un dato asíncrono, concluimos, no puede delegarse al modelo de ejecución, al estilo sintáctico elegido para consumirlo, ni siquiera a un `try/catch` bien colocado.

REPOSITORIO

El código fuente completo de las tres implementaciones, el módulo de post-procesamiento y el caso límite está disponible en: <https://github.com/MimDodge/First-Laboratory-Multimedia-Data-Bases-.git>.

REFERENCIAS

- [1] S. A. Reyes Gómez, “Laboratorio: Javascript Avanzado,” Facultad de Ingeniería, Universidad de San Buenaventura, Bogotá, Colombia, 2026.
- [2] Mozilla Developer Network, “Promise,” MDN Web Docs. [En línea]. Disponible: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise
- [3] Mozilla Developer Network, “Modelo de concurrencia y bucle de eventos,” MDN Web Docs. [En línea]. Disponible: https://developer.mozilla.org/es/docs/Web/JavaScript/Event_loop
- [4] Ecma International, “ECMAScript 2015 Language Specification,” 6.^a ed., Jun. 2015. (Especificación original de `Promise`).
- [5] Ecma International, “ECMAScript 2017 Language Specification,” 8.^a ed., Jun. 2017. (Especificación original de `async/await`).
- [6] OpenJS Foundation, “Timers,” Node.js Documentation. [En línea]. Disponible: <https://nodejs.org/api/timers.html>